

Heuristic algorithms for tree-depth of social networks

Chia-Hao Lu, Bang Ye Wu

Department of Computer Science and Information Engineering
National Chung Cheng University
Chiayi, 621 Taiwan

Abstract

The tree-depth of a graph G is the minimum height of any rooted forest F such that $G \subseteq \text{closure}(F)$. We implemented a dynamic programming algorithm for finding the minimum tree-depth and designed heuristic algorithms by using betweenness, closeness, triangle, and degree centralities. We also proposed a heuristic algorithm based on the tree-depth decomposition tree for the graph coloring problem. We studied some cases that our heuristic algorithm is much better than the degree first method.

1 Introduction

In this paper, we study heuristic algorithms for computing the tree depth of social networks and develop heuristic algorithms for solving the minimum coloring problem based on tree-depth decompositions.

A graph G consists of a vertex set V and an edge set E . Two vertices are adjacent if there is an edge between them. In this paper, edges are undirected and unweighted. A rooted tree T is a connected graph without cycle such that there is a specified vertex as the root of T . The height of T is the distance from the root to the farthest vertex in T . A rooted forest F is composed of one or more rooted trees, and the height of F is the maximum height of the rooted trees in F . The closure of F , denoted by $\text{closure}(F)$, is a graph with the same vertex set such that every vertex is adjacent to all of its ancestors in F . For a given graph G , a rooted forest F is a feasible “tree-depth” decomposition of G if $G \subseteq \text{closure}(F)$. The (optimal) tree-depth of G , denoted by $\text{td}(G)$, is the minimum height of any feasible tree-depth decomposition of G , and the (optimal) tree-depth decomposition of G is the one achieves with minimum height. In other words, an optimal tree-depth decomposition is a minimum height forest such that the edges only appear between a vertex and its ancestor. For simplicity, we shall use “decomposition tree” instead of “tree-depth decomposition tree”.

For example, in Figure 1, graph (a) is the original graph with four vertices and four edges. Both graph (b) and graph (c) are decompositions

of graph (a) with the height four and three respectively. In this case, we cannot find a tree-depth decomposition with the height smaller than three, so the tree-depth of (a) is three, and (c) is an optimal tree-depth decomposition of the original graph (a).

Previous studies defined tree-depth and presented an inductive form of tree-depth (as Lemma 1.) in [7]. It is an NP-complete problem to find the optimal minimum tree-depth. The time complexity of finding a minimum tree-depth is $O(2^n)$ by enumerating all possibility of decompositions with dynamic programming. Fomin et al. presented an algorithm with time complexity $O(1.9602^n)$ in [2].

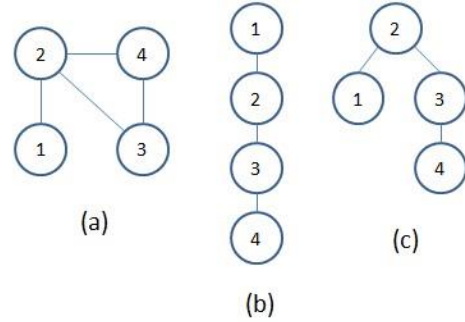


Figure 1. An example of tree-depth and its decompositions. (a) is the original graph. (b) and (c) are both decompositions of (a).

Lemma 1. Let G be a graph, and that $G_1, \dots, G_p$ be its connected components. Then :

$$\text{td}(G) = \begin{cases} 1, & \text{if } |V(G)| = 1 \\ 1 + \min_{v \in V(G)} \text{td}(G - v), & \text{if } p = 1 \text{ and } |V(G)| > 1 \\ \max_{i=1}^p \text{td}(G_i), & \text{otherwise} \end{cases}$$

To find a decomposition of a connected graph, one chooses some vertices one by one until the graph is cut into components. Then the process is recursively performed on the components until there is no edge. The chosen vertices form a path in the decomposition tree and the decomposition trees of the components are subtree of it. Intuitively, to find a decomposition tree with smaller height, we try to remove vertices to divide the graph more “evenly”. It is similar to the hierarchical community detection in social

networks. To find a possibly better decomposition of social networks, our intuition chooses more important vertices defined in social networks. In social networks, a person is likely to be important if he/she has some properties or centrality, such as degree, closeness, or betweenness. We used these centralities to find a suboptimal tree-depth and its decomposition in short time. To show the qualities of the solution, we performed experiments.

In our experiments, the tree-depths utilizing betweenness and closeness centralities are close to the minimum value. We also used the triangle number of vertices, which is the number of edges between neighbors of a vertex, to compute the tree-depth. It performs well in artificial networks, but less well in some real data networks.

The second part of our study is to develop heuristic algorithms for solving hard problems by using decomposition trees. In this paper, we concerned on the graph coloring problem. The graph coloring problem asks the minimum number of colors to color the vertices of a graph such that any two adjacent vertices are of different colors. Finding the minimum number of colors, i.e. the chromatic number, is a well-known and important NP-complete problem. An intuitive greedy method iteratively selects the vertex with the highest degree and tries to color it with a used color. If it is impossible, then a new color will be used. By definition, the height of any decomposition tree is an upper bound of the chromatic number. Thus, a straight forward idea is to color a graph with the help of a decomposition tree.

We used tree-depth decomposition to compute the heuristic graph coloring. We listed some cases that our method is better than the degree method. In one of the extreme cases, the number of colors we used is three, but it could be up to n by degree method, where $2n$ is the number of vertices in the bipartite graph.

The paper is organized as follows. In Section 2 we introduce the tree-depth and some social network centralities we used in computing tree-depth. In Section 3 we present our experiment results. Section 4 is a heuristic method to solve graph coloring problem with tree-depth decompositions. Section 5 is the conclusion.

2 Tree-depth and social network centralities

We can obtain a tree-depth decomposition by selecting a vertex at every turn from the given graph. If the graph is broken after removing the selected vertex, then the decomposition branches out. For example, In Figure 1, the orderings of selecting vertices are 1, 2, 3, 4 and 2, 1, 3, 4 for (b) and (c) respectively. When we change the ordering

of selecting vertices, we get the different decompositions. The minimum tree-depth decomposition can be found by enumerating all possibility of decompositions with dynamic programming.

In some times, we do not need to get the minimum tree-depth. We wanted to get a suboptimal tree-depth in short time, so we selected the vertex with different social network centralities at every turn. The following is an introduction to the centralities we used in this paper.

Degree centrality:

$$D(v) = |N(v)| \quad (1)$$

The degree of a vertex v is the number of neighbors of v . In social networks, it means a person is powerful with high degree, because he/she can influence more people. We first select the vertex with the highest degree at every turn in our studies. If the number of vertices with the highest degree is more than one, we select one of them randomly. The time complexity of degree centrality is $O(V^2)$ at the beginning. It is $O(V)$ to update the degree after removing a vertex.

Closeness centrality:

$$C(x) = \frac{1}{\sum_y d(y,x)} \quad (2)$$

For a vertex x in a connected graph, the closeness of x is the inverse of sum of the distances from x to all other vertices in the graph. In social networks, it means a person can reach all other people faster than others with high closeness. We first select the vertex with the highest closeness at every time in our studies. The time complexity of closeness centrality is $O(VE)$ for selecting a vertex.

Betweenness centrality:

$$B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (3)$$

For vertices s , v , and t ($s \neq v \neq t$) in a connected graph G . σ_{st} is the number of all shortest paths between s and t . $\sigma_{st}(v)$ is the number of the shortest paths between s and t passing through the vertex v . The betweenness of v is the sum of the ratio of $\sigma_{st}(v)$ to σ_{st} for all s and t . In social networks, it means a person plays the role of bridge between others with high betweenness. The vertex with the highest betweenness will be selected first at every turn in our studies. We used the algorithm Brandes proposed in [1] to compute betweenness centrality. The time complexity of betweenness centrality is

$O(VE)$ for selecting a vertex.

Triangle:

$$T(x) = |\{e_{jk}: v_j, v_k \in N_i, e_{jk} \in E\}| \quad (4)$$

For a vertex i , the triangle of i is the number of edges between the neighbors of i . The time complexity of triangle is $O(V^3)$ at the beginning. It is $O(V)$ to update the triangle after removing a vertex. We observed betweenness centrality and closeness centrality perform well in cycle and path graphs. By removing all triangles rapidly, we got the graph that only contains cycles and paths. Then, we used betweenness centrality or closeness centrality to handle the remaining graph. It possibly can reduce the computation time, and perform as well as betweenness and closeness.

3 Experiments

In this section, we show the experiment results of computing the tree-depths with different centralities in some networks. We used some artificial networks and real data networks.

In artificial networks, we generated ten graphs on ER model, generative model, and power-law model respectively. In ER model, we generated an undirected random graph with 20 vertices. For each graph, it is 10% probability to generate an edge between any two vertices. Table 1 shows the results in ER model. The first row is the number of test graphs from 1 to 10. DP is the minimum tree-depth computed by dynamic programming. B is for the betweenness method. C is for the closeness method. Tb is for the triangle method with betweenness. Tc is for the triangle method with closeness. D is for the degree method. In generative model, the initial graph is three isolated vertex. We add a vertex with three edges at every turn, and distribute edges to the current vertices by their degree distributions until the whole graph contains 300 vertices. Table 2 shows the results in generative model. We computed tree-depth by randomly selecting vertex 100 times. R is the minimum value in the 100 times. In power-law model, it contains about 300 vertices. The degree and the number of degree are in inverse proportion. Table 3 shows the results in power-law model. Most of all, betweenness, closeness, and triangle perform better than degree and random in artificial networks. As expected, triangle performs as well as betweenness, and just cost half of time the betweenness used.

We also did experiments on five real data networks, karate [8], dolphins [5], adjnoun [6], football [3], and Les Miserables [4]. Table 4 shows the results with different centralities. N is the number of vertices, and M is the number of edges. Among them, karate, dolphins, and football

are social networks. Obviously, betweenness and closeness still perform well in real data networks. To our surprise, triangle performs less well in dolphins and football, although it is still as good as degree. The reason is that there are many vertices with the most triangles in the same time. Therefore, it could select a bad vertex for computing tree-depth in this situation. It can be solved by considering other conditions, like the degree of neighbors. To sum up, using betweenness centrality or closeness centrality to compute a suboptimal tree-depth performs well whether in artificial networks or in real data networks.

Table 1: The results in ER model.

	1	2	3	4	5	6	7	8	9	10
DP	6	5	5	5	5	6	6	7	7	4
B	8	5	5	5	6	6	6	8	7	4
C	7	5	5	5	6	6	6	7	7	4
Tb	6	5	5	5	7	6	6	7	7	6
Tc	6	5	5	5	6	6	6	7	7	6
D	8	5	6	5	6	6	6	9	7	6

Table 2: The results in generative model.

	1	2	3	4	5	6	7	8	9	10
B	71	69	65	67	63	67	67	65	72	59
C	73	67	62	68	64	71	67	65	73	57
Tb	72	69	62	68	64	68	65	67	71	57
Tc	74	70	64	68	63	69	65	68	74	61
D	86	75	69	79	69	78	75	79	84	65
R	196	198	203	194	192	199	204	198	175	202

Table 3: The results in power-law model.

	1	2	3	4	5	6	7	8	9	10
B	16	18	17	18	19	19	23	21	16	17
C	16	20	18	17	19	21	22	23	16	19
Tb	16	19	18	18	19	19	24	21	16	17
Tc	17	19	18	18	19	19	23	21	16	16
D	20	24	22	17	19	20	24	24	19	23
R	111	102	96	134	120	117	137	134	115	99

Table 4: The results of five real data networks.

	karate	dolphins	adjnoun	football	Les
N	34	62	112	115	77
M	78	159	425	616	254
B	8	18	39	55	18
C	8	17	39	52	16
Tb	8	22	40	82	22
Tc	8	22	38	83	22
D	9	23	44	76	16
R	16	34	74	89	28

4 Applications with tree-depth

The graph coloring problem asks the minimum number of colors to color the vertices of

a graph such that any two adjacent vertices are of different colors. An intuitive greedy method iteratively selects the vertex with the highest degree and tries to color it with a used color. If it is impossible, then a new color will be used. Instead of selecting the vertex with the highest degree, our heuristic algorithm is selecting the top vertex of the decomposition tree at every turn. We studied some cases that our heuristic algorithm is better than the degree method in the following.

Figure 2 shows a bipartite graph that contains eight vertices. In the degree method, the vertex 2, 3, 5, and 7 are with the highest degree after selecting vertex 1. If we select vertex 2 in the next step, then it needs additional one color to use. In the worst case, it needs four colors for this graph. In our method, we used the decomposition tree of dynamic programming to select vertex 1, 3, 5, and 7 step by step. These four vertices can be of the same color because there is no edge between them. It needs only two colors (optimal solution) for this graph. Figure 3 shows the results of two methods. When generalized to n vertices in each partite, the number of colors found by the degree method on the bipartite graph will be increased to n , and the ratio of our method to the degree method could be 2 to n .

Figure 4 shows another bipartite graph that adds two adjacent vertices to vertex 1, 3, 5, and 7 respectively, and two non-adjacent vertices to vertex 2, 4, 6, and 8 respectively. In this graph, the degree method could use four colors in worst case. In our method, we used the decomposition tree of betweenness and triangle to select vertex 1, 3, 5, and 7 step by step. These four vertices can be of the same color because there is no edge between them. It needs three colors (optimal solution) for this graph. When generalized to n vertices in each partite, the number of colors found by the degree method on the bipartite graph will be increased to n , and the ratio of our method to the degree method could be 3 to n .

Figure 5 shows a 3-cube. In the degree method, the vertex 4, 6, 7, and 8 are with the highest degree after selecting vertex 1. It needs additional one color to use if we select vertex 8 in the next step. In the worst case, it needs four colors for this graph. In our method, it obtains the optimal solution (two colors) with the decomposition tree of dynamic programming. Figure 6 shows the results of two methods.

Except for graph coloring problem, tree-depth is helpful to other problems, for example, dominating set, vertex cover, and so on.

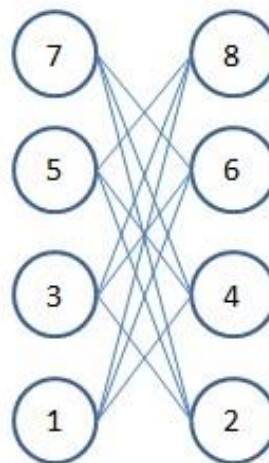


Figure 2

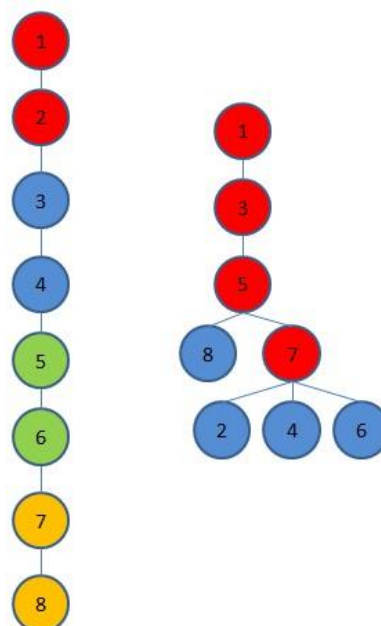


Figure 3: The results in Figure 2 with the degree method (left) and our method (right).

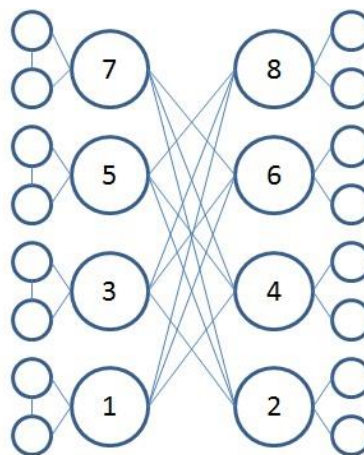


Figure 4

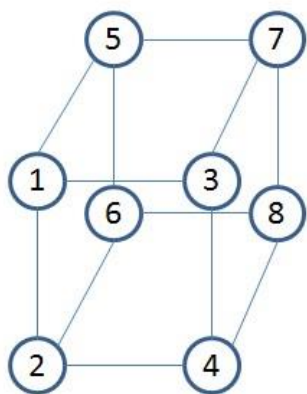


Figure 5

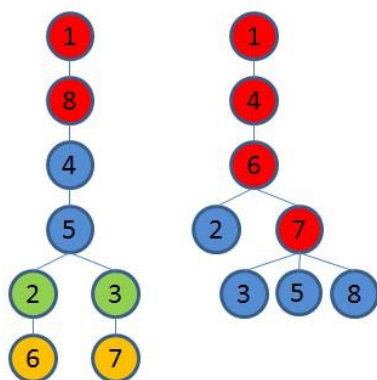


Figure 6: The results in Figure 5 with the degree method (left) and our method (right).

5 Conclusion

In this study, we implemented the dynamic programming algorithm for finding the minimum tree-depth and designed heuristic algorithms by using betweenness, closeness, triangle, and degree centralities. By our experiments, it shows that the solutions found by betweenness or closeness are better than the ones by degree. Triangle method performs similarly to the betweenness method and uses less running time for the artificial networks. But it performs worse on real data networks.

We proposed a heuristic algorithm based on decomposition tree for the graph coloring problem. For some extreme cases, our method is much better than the degree method. Our future work is to investigate if tree-depth is helpful to solve other problems, like dominating set or vertex cover.

References

- [1] Brandes, U. (2001). A faster algorithm for betweenness centrality*. *The Journal of Mathematical Sociology*, 25(2), 163–177.
- [2] Fomin, F. V., Giannopoulou, A. C., & Pilipczuk, M.

(2014). Computing tree-depth faster than 2^n . *Algorithmica*, 73(1), 202–216.

- [3] Girvan, M., & Newman, M. E. J. (2002). *Community structure in social and biological networks* (Vol. 99)
- [4] Knuth, D. E. (1993). *The Stanford GraphBase: a platform for combinatorial computing* (Vol. 37). Reading: Addison-Wesley.
- [5] Lusseau, D., & Newman, M. E. (2004). Identifying the role that animals play in their social networks. *Proceedings of the Royal Society of London B: Biological Sciences*, 271(Suppl 6), S477-S481.
- [6] Newman, & J, M. E. (2006). Title: *Finding community structure in networks using the eigenvectors of matrices*.
- [7] Nešetřil, J., & Ossona de Mendez, P. (2006). Tree-depth, subgraph coloring and homomorphism bounds. , 27(6), 1022 – 1041.
- [8] Zachary, W. W. (1977). *An information flow model for conflict and fission in small groups* (Vol. 33).